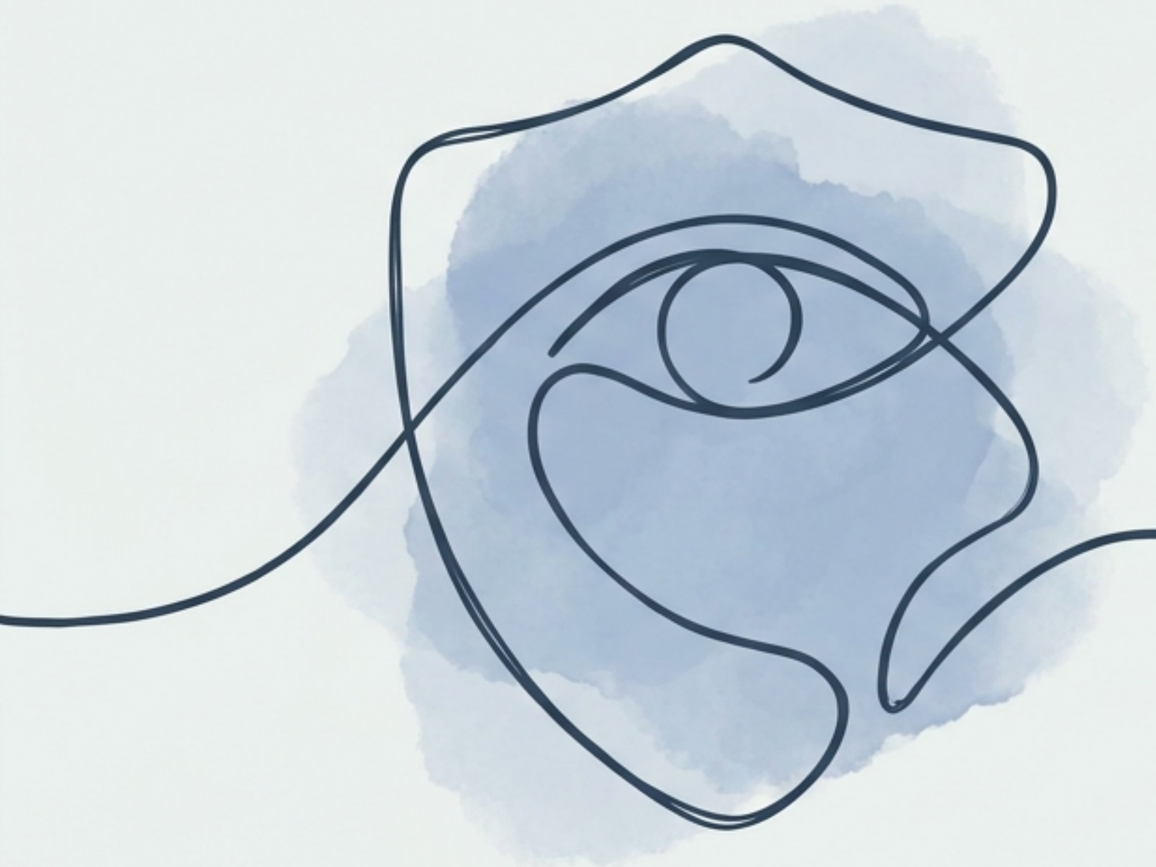
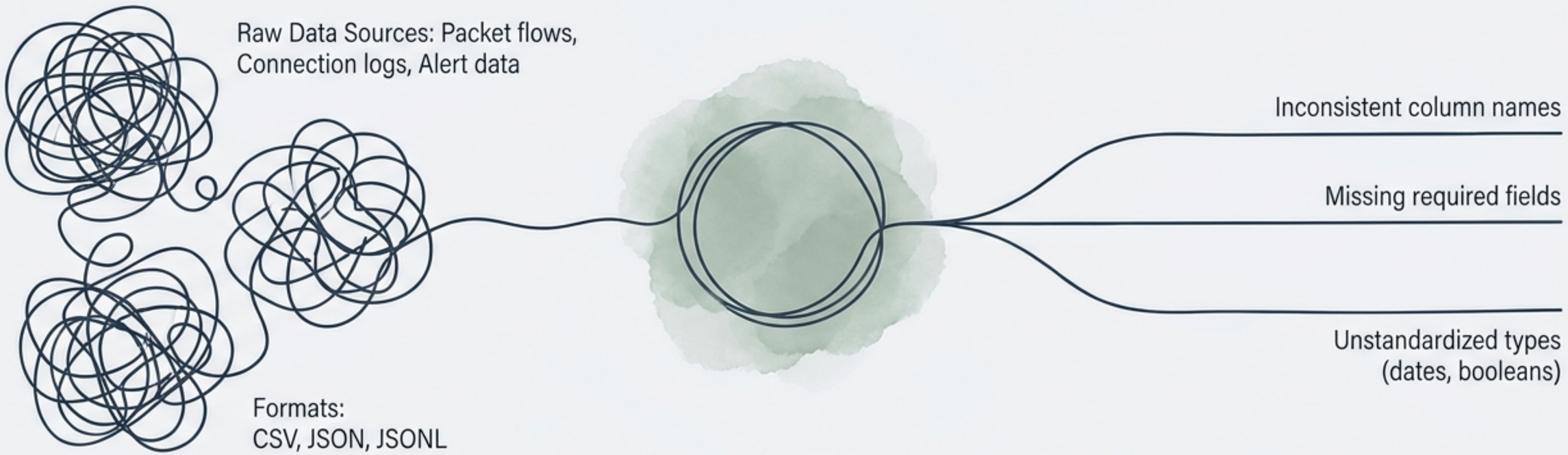


Final Project: DataGuard

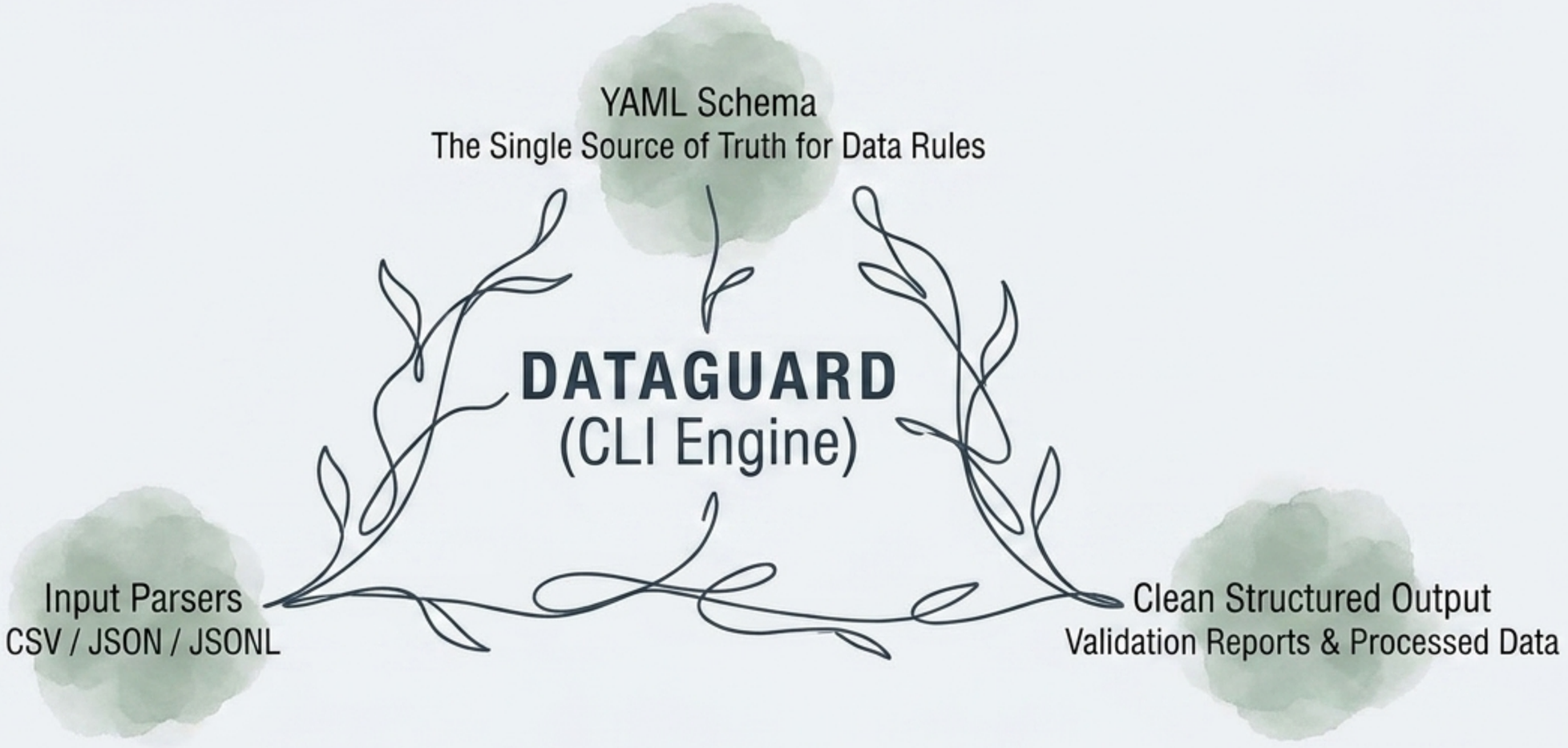
Schema-Driven Data Validation, Cleaning, and Conversion CLI

The Challenge of Fragmented Truth



Manual cleanup is time-consuming and prone to human error.

The DataGuard Solution Framework



Automatically parse, inspect, and organize structural data from multiple sources through a strict, unified configuration.

The Three Core Pathways

Validate

- Formula: Input + Schema -> JSON Report
- Action: Checks required fields, string patterns, and boundaries.
- Output: Comprehensive JSON validation and parse error report.

Clean

- Formula: Input + Schema + Transforms -> Clean CSV + Report
- Action: Applies type casting, fills missing data, and deduplicates. Filters invalid rows.
- Output: Output payload containing only validated rows, plus a JSON report.

Convert

- Formula: Format A -> Format B
- Action: Pure format translation between CSV, JSON, and JSONL.
- Output: Strict data serialization without schema validation or transformation.

Architecture Progression Timeline

Week 7 (Scaffold)

Established core CLI infrastructure and base validation flow.

Week 8 (Validation Layer)

Implemented strict rules, unified error reporting, and schema enforcement.

Week 9 (Transformation Layer)

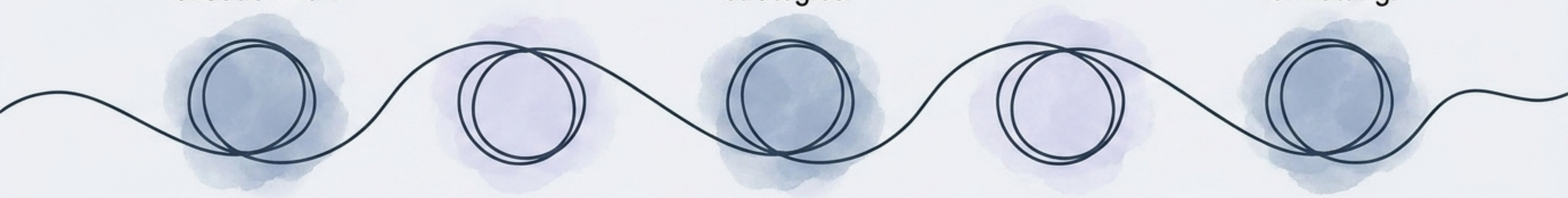
Decoupled data transformation engine and operational strategies.

Week 10 (End-to-End Orchestration)

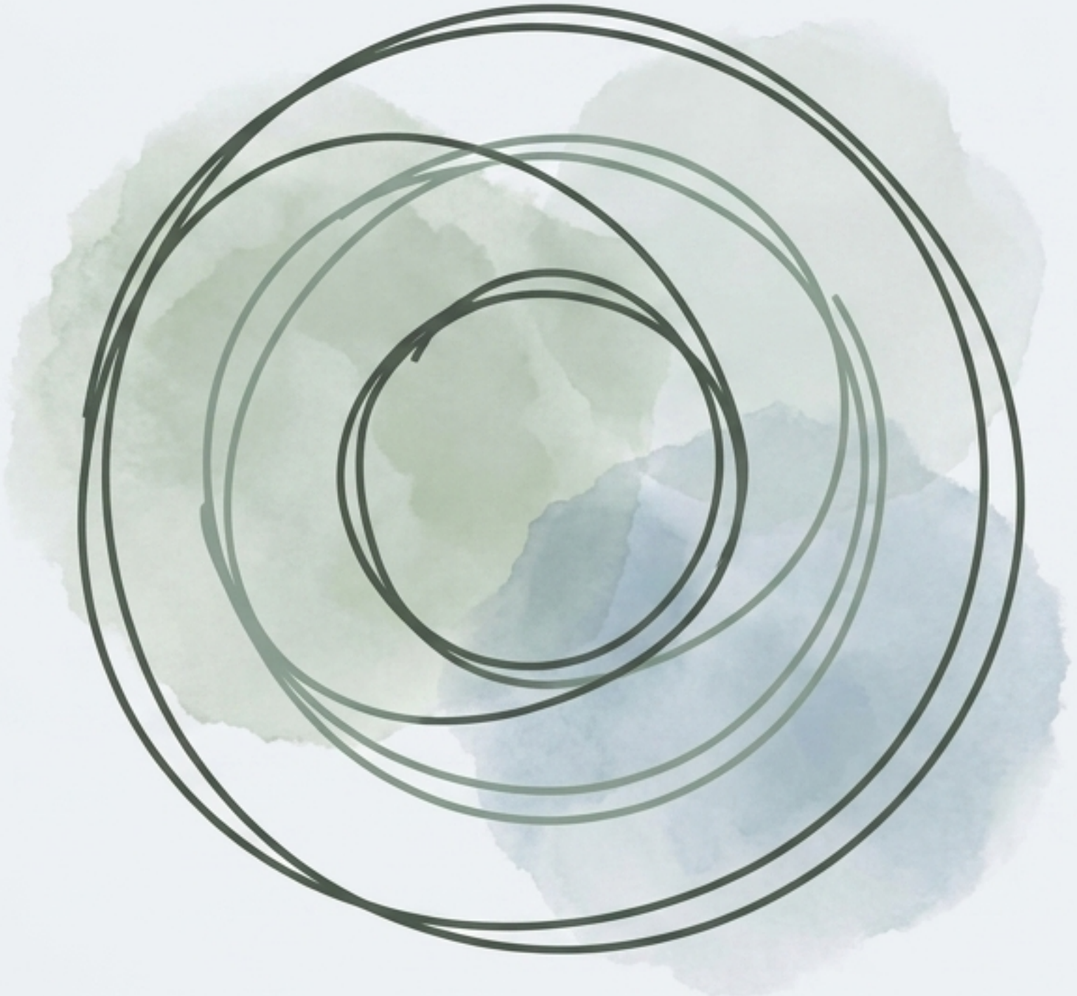
Combined transforms and validation into the complete ‘clean’ CLI workflow.

Week 11 (Format Flexibility)

Finalized output writer factories and pure ‘convert’ formatting.



Deep Dive: Rigorous Validation Engine



Outer Ring: Strict Schema Enforcement

strict: true flag deployed to instantly identify and report UNKNOWN_COLUMN inputs.

Middle Ring: Expanded Data Rules

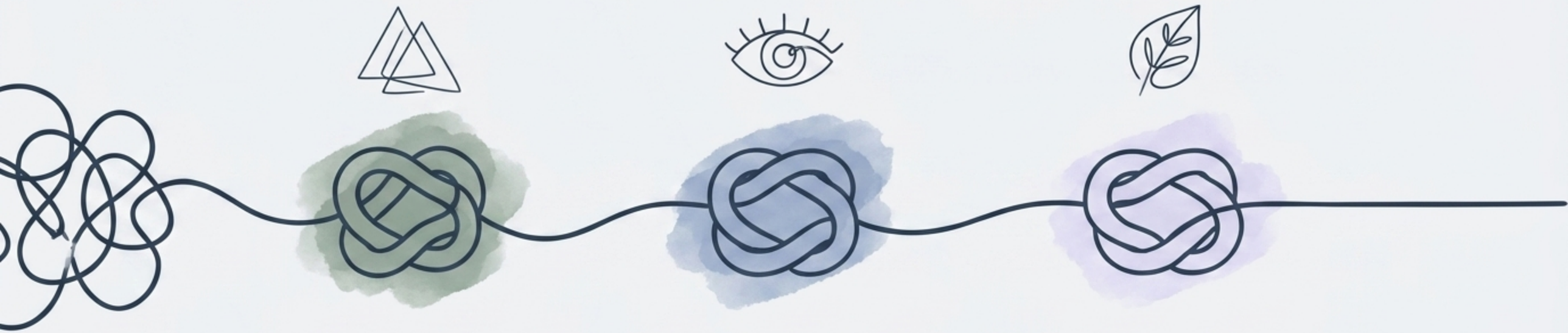
Advanced evaluators for min/max string length, enums, booleans, and strict date formatting.

Inner Ring: Unified Reporting

Parse errors and validation errors are combined into a single, comprehensive JSON report summarizing total error counts.

Deep Dive: The Transformer Foundation

An ordered execution engine driven by YAML configurations, operating entirely decoupled from file I/O.



type_cast

Safely converts integers, floats, strings, and booleans with fallback retention for failed parses.

fill_missing

Executes defined strategies to either inject default values or trigger drop_row for incomplete records.

dedup

Targeted deduplication using specific composite keys, supporting both keep=first and keep=last strategies.

Deep Dive: Seamless Orchestration & Output

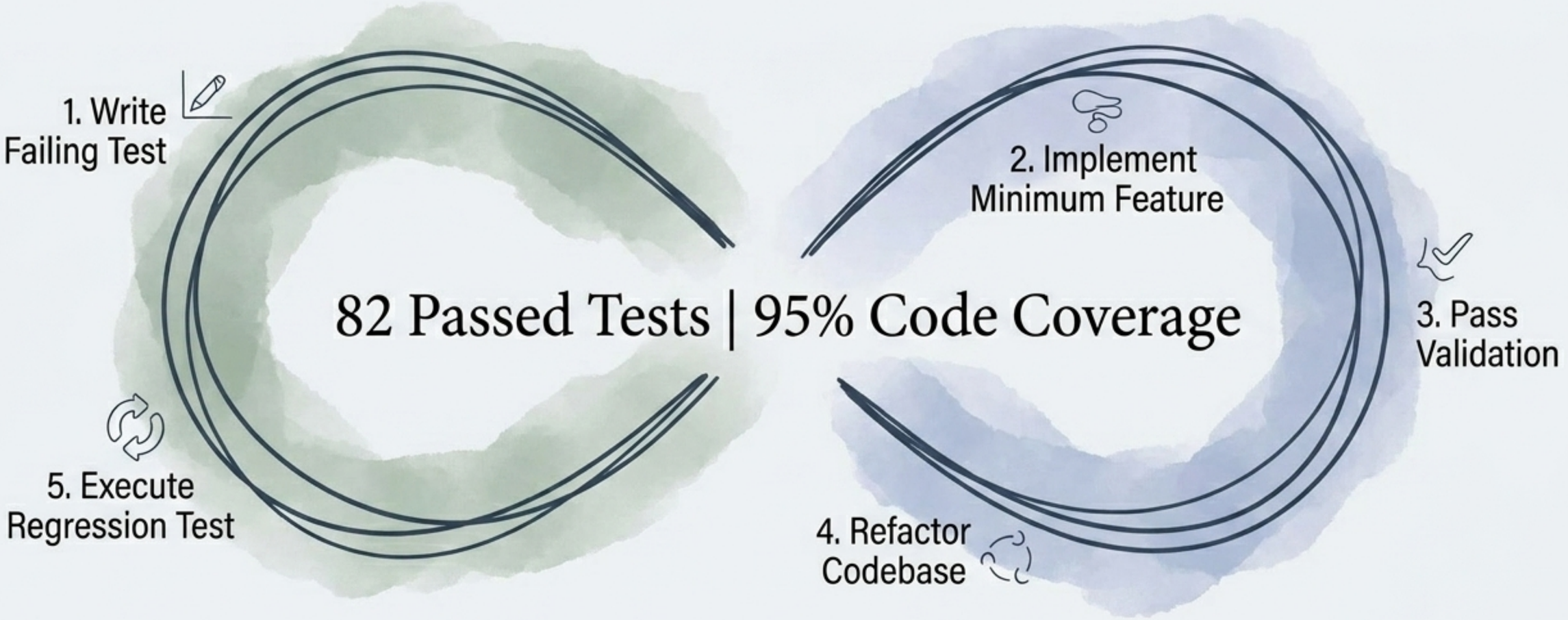
The Clean Flow

- Loads YAML transforms, applies data rules, and filters invalid rows.
- Maintains CLI semantic correctness (e.g., exiting with code 1 if errors persist).

The Output Writer Factory

- Dynamic routing based on file extensions.
- Seamlessly serializes parsed records to .csv, .json (formatted array payload), or .jsonl (line-delimited objects).
- Strict error handling for missing inputs or unsupported formats.

Engineering Craftsmanship: Test-Driven Development



Coverage spans granular parser unit tests, CLI contract tests, input space partitioning (valid/invalid/edge cases), and full end-to-end integration flows.

The Horizon: Domain-Specific Application

Cyber Schemas

Developing out-of-the-box YAML templates for network flows, connection logs, and IOCs.

Intelligent Field Mapping

Auto-normalizing fragmented headers (e.g., merging src_ip, source_ip, and SourceAddress into a single unified column).

Enhanced Reporting

Elevating the JSON output to include human-readable summaries and severity grouping.

Realistic Threat-Hunting Demos

Providing packaged demo datasets to simulate raw cyber data ingestion, automated cleaning, and finalized report generation.